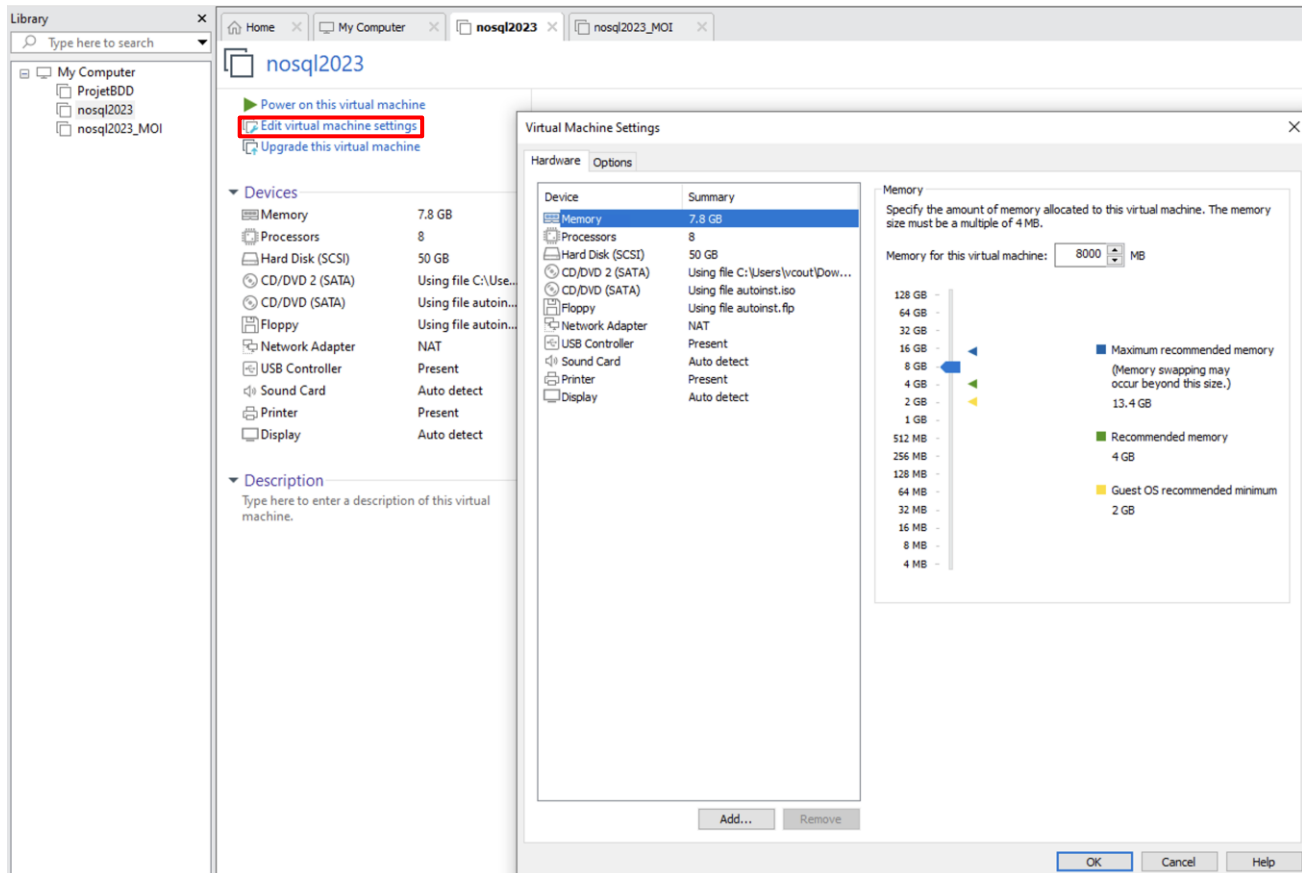


TP MOTEUR DE RECHERCHE - ELK

Machine virtuelle VMware :

- Copier le dossier `U:\VM\INFO\Linux-Ubuntu\nosql2023_VMWARE` sur le disque D
- Exécuter le fichier `vmx`
- La machine virtuelle sera alors chargée dans VMware Workstation
- Vous pouvez modifier les paramètres (RAM, CPU) :



- Lancer la VM
- Login / MdP : `nosql` / `nosql2023`

Comment lire ce TP ?

Ceci est une question à réaliser

Ceci est un morceau de code ou une commande Bash à exécuter

Ceci est un cadre contenant des informations utiles

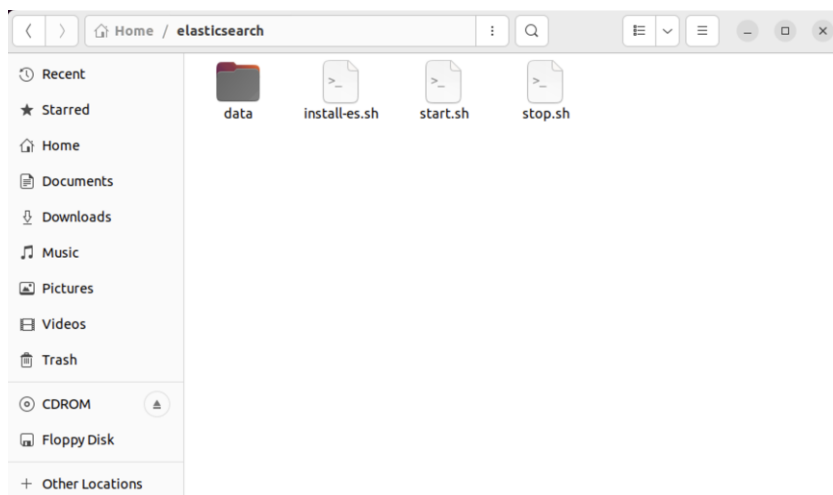
Ceci est un cadre avec des informations importantes

BIEN CONSERVER L'ENSEMBLE DE VOS CODES EN CAS DE PROBLEME AVEC LA VM

INTRODUCTION

Le but de ce TP est de découvrir et prendre en main la suite ELK avec une installation rapide, une indexation de gros volumes de données et d'effectuer des recherches & statistiques sur cette base. Pour cela nous allons utiliser deux CSV exportés d'une base de données de séries TV (Base de données à jour au 01/01/2018, soit 16276 séries pour 862481 épisodes + 1 ligne dans chaque fichier pour le header).

Le répertoire Elasticsearch du répertoire personnel (*/home/nosql/elasticsearch*) sera notre répertoire principal de travail.



Pour commencer, ouvrir le fichier `install-es.sh` se trouvant dans le dossier `/home/nosql/elasticsearch`. Il permet de télécharger et d'installer l'ensemble des logiciels (ES, Kibana, Logstash, etc.).

Ici, le cluster Elasticsearch sera nommé « `but3-tp` » :

```

1 #!/bin/bash
2 echo "#####"
3 echo "### Downloading ES ###"
4 echo "#####"
5
6 wget https://artifacts.elastic.co/downloads/elasticsearch/elasticsearch-8.10.1-linux-x86_64.tar.gz
7 tar -xvzf elasticsearch-8.10.1-linux-x86_64.tar.gz
8 mv elasticsearch-8.10.1 elasticsearch
9 rm -f elasticsearch-8.10.1-linux-x86_64.tar.gz
10
11 echo "#####"
12 echo "### Downloading Logstash ###"
13 echo "#####"
14
15 wget https://artifacts.elastic.co/downloads/logstash/logstash-8.10.1-linux-x86_64.tar.gz
16 tar -xvzf logstash-8.10.1-linux-x86_64.tar.gz
17 mv logstash-8.10.1 logstash
18 mkdir logstash/conf
19 touch logstash/conf/csvload.conf
20 rm -f logstash-8.10.1-linux-x86_64.tar.gz
21
22 echo "#####"
23 echo "### Downloading Kibana ###"
24 echo "#####"
25
26 wget https://artifacts.elastic.co/downloads/kibana/kibana-8.10.1-linux-x86_64.tar.gz
27 tar -xvzf kibana-8.10.1-linux-x86_64.tar.gz
28 mv kibana-8.10.1-linux-x86_64 kibana
29 rm -f kibana-8.10.1-linux-x86_64.tar.gz
30
31 echo "#####"
32 echo "### Configuring ES ###"
33 echo "#####"
34
35 sed -i -e 's/#cluster.name: my-application/cluster.name: but3-tp/g' elasticsearch/config/elasticsearch.yml
36 echo "OK"
37
38 echo "#####"
39 echo "### Downloading Cerebro ###"
40 echo "#####"
41
42 wget "https://github.com/lmenezes/cerebro/releases/download/v0.9.4/cerebro-0.9.4.zip"
43 unzip cerebro-0.9.4.zip
44 mv cerebro-0.9.4 cerebro
45 rm -f cerebro-0.9.4.zip
46 echo "OK"
47
48 echo "Install successful !"

```

Lancer le script via le terminal :

(NE PAS FAIRE L'INSTALLATION AVEC sudo)

```
sh install-es.sh
```

1. Ingestion des données

Logstash va être notre ETL pour lire, transformer et envoyer nos données CSV vers Elasticsearch. Ayez toujours un onglet ouvert sur la documentation de Logstash : <https://www.elastic.co/guide/en/logstash/8.10/index.html>

Un fichier de configuration dans le dossier conf de Logstash a été créé (logstash/conf/csvload.conf). C'est dans ce dossier que doivent être stockés les différents fichiers de configuration. Ces fichiers sont vérifiés toutes les 3s pour intégrer les modifications apportées. N'oubliez donc pas de sauvegarder.

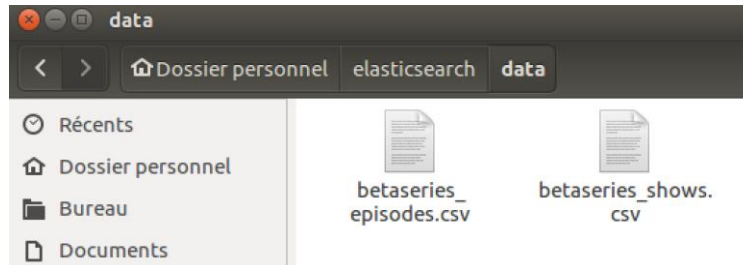
Pour lancer Logstash avec un fichier de configuration, il suffit d'exécuter cette commande depuis /elasticsearch :

```
logstash/bin/logstash -f /home/nosql/elasticsearch/logstash/conf/<VotreFichierDeConf>
```

Si vous exécutez la commande, vous obtiendrez, pour l'instant, des erreurs, puisque le contenu du fichier conf est vide.

Logstash met beaucoup de temps à démarrer, d'autant plus dans une VM. Soyez patient !
Pour arrêter Logstash, un simple Ctrl+C suffit pour lui envoyer un signal d'extinction

N'hésitez pas à ouvrir les deux fichiers CSV du dossier Data et regarder leur contenu.



- a) Première configuration composée d'un input et d'un output qui lisent les fichiers contenus dans le répertoire elasticsearch/data et les affichent sur le terminal. Modifier le contenu du fichier csvload.conf.

Code :

```
# Fichier de configuration logstash
input {
  file {
    # chemin d'accès
    path => "/home/nosql/elasticsearch/data/"
    # curseur de lecture à supprimer pour relancer
    sincedb_path => "/dev/null"
    # Lecture depuis le début
    start_position => "beginning"
  }
}

filter{
}

output {
  # affichage sur la sortie standard
  stdout {
    codec => "rubydebug"
  }
}
```

Explications :

- Nous utilisons les plugins file en input et stdout en output (avec un codec rubydebug pour plus de lisibilité) :
 - o <https://www.elastic.co/guide/en/logstash/8.10/plugins-inputs-file.html>
 - o <https://www.elastic.co/guide/en/logstash/8.10/plugins-outputs-stdout.html>
- *: pour sélectionner les deux fichiers en même temps dans le path
- Le plugin file commence à lire un fichier par la fin par défaut. Ici, nous lui indiquons de commencer par le début (start_position => "beginning").
- Le plugin file comporte un paramètre sincedb qui permet de garder une trace de son exécution. Il faudra soit supprimer le fichier .since_db à chaque fois soit le rediriger vers /dev/null, comme nous l'avons fait.
- Pour le moment, aucune transformation n'est réalisée (plugin filter : <https://www.elastic.co/guide/en/logstash/8.10/core-operations.html>)

Résultat (après avoir relancé Logstash) : chaque ligne des fichiers s'affiche dans le terminal (les champs ne seront pas forcément dans le même ordre)

```

nosql@nosql-virtual-machine: ~/elasticsearch
{
  "event" => {
    "original" => "268827,3639,\"Home and Away\", \"S14E47\",14,47,\"Episode 3007\", \"Vinnie has a crisis of confidence. Jade suspects Brodie has feelings for Alex. Noah rescues Toni from the irate farmer Leo. Nick wants to spend time with Kirsty alone, but she's giving him the brush-off.\",985042800"
  },
  "log" => {
    "file" => {
      "path" => "/home/nosql/elasticsearch/data/betaserie_episodes.csv"
    }
  },
  "@timestamp" => 2023-11-13T18:29:23.524699045Z,
  "message" => "268827,3639,\"Home and Away\", \"S14E47\",14,47,\"Episode 3007\", \"Vinnie has a crisis of confidence. Jade suspects Brodie has feelings for Alex. Noah rescues Toni from the irate farmer Leo. Nick wants to spend time with Kirsty alone, but she's giving him the brush-off.\",985042800",
  "host" => {
    "name" => "nosql-virtual-machine"
  },
  "@version" => "1"
}
{
  "event" => {
    "original" => "268828,3639,\"Home and Away\", \"S14E48\",14,48,\"Episode 3008\", \"Nick gives Kirsty an ultimatum - single dates or else. Alex seeks Brodie's help in getting closer to Dani, much to Brodie's heartbreak. Vinnie feels he can do no right with Leah. Gypsy makes a life-changing discovery.\",985129200"
  },
  "log" => {
    "file" => {
      "path" => "/home/nosql/elasticsearch/data/betaserie_episodes.csv"
    }
  },
  "@timestamp" => 2023-11-13T18:29:23.524704437Z,
  "message" => "268828,3639,\"Home and Away\", \"S14E48\",14,48,\"Episode 3008\", \"Nick gives Kirsty an ultimatum - single dates or else. Alex seeks Brodie's help in getting closer to Dani, much to Brodie's heartbreak. Vinnie feels he can do no right with Leah. Gypsy makes a life-changing discovery.\",985129200",
  "host" => {
    "name" => "nosql-virtual-machine"
  }
}

```

Vous pouvez utiliser CTRL+C pour arrêter l'exécution.

On remarque les champs :

- message : contient toutes les données concaténées que l'on souhaite récupérer
- [event][original] : même contenu que message
- [log][file][path] : chemin vers le fichier contenant la ligne
- @timestamp : date actuelle
- Etc.

b) Maintenant, on va faire comprendre à Logstash que le fichier est un CSV à interpréter. Rajouter dans filter un plugin csv qui va parser nos données. La première ligne du CSV contient le nom des colonnes. Il faut ensuite supprimer le champ message qui contient la ligne CSV et ne sera pas utile.

Indications :

- Vu qu'on a deux formats différents de csv, il faut utiliser IF / ELSE (<https://www.elastic.co/guide/en/logstash/8.10/event-dependent-configuration.html>) avec éventuellement IN (pour utiliser le champ, il faudra le mettre entre [], exemple : IF

[monchamp]="valeur") et donner manuellement les colonnes du CSV à la configuration des deux plugins CSV (<https://www.elastic.co/guide/en/logstash/8.10/plugins-filters-csv.html>) .

- Pour avoir accès à path, il faudra utiliser [log][file][path]

- Les colonnes sont fournies en annexes.

- Le plugin `mutate` permet de modifier un objet (ajout/suppression de champ, modification de valeur, ...). Utiliser l'option `remove_field` : <https://www.elastic.co/guide/en/logstash/8.10/plugins-filters-mutate.html>.

Exemple de code pour la transformation des données des séries :

```
input {
  file {
    path => "/home/nosql/elasticsearch/data/betaserie_shows.csv"
    sincedb_path => "/dev/null"
    start_position => "beginning"
  }
}
filter{
  # on parse le fichier shows
  csv {
    columns => [
      "show_id","title","description","seasons","episodes","follow_count","creation_year","genres","network"
    ]
    skip_empty_rows => true
    separator => ","
    skip_header => true
  }
}
output {
  stdout {
    codec => "rubydebug"
  }
}
```

Résultats : Le résultat en stdout s'affiche maintenant découpé en champs du nom des colonnes du CSV dans un format attendu (JSON, RUBY, ...).


```
{
  "log" => {
    "file" => {
      "path" => "/home/nosql/elasticsearch/data/betaserie_shows.csv"
    }
  },
  "follow_count" => "4",
  "message" => "8323,\"Pic Pic et André\",\\\"Le Pic Pic André Shoow est une série de court métrages d'animation de quatre épisodes (Un premier en 1988 intitulé Picpic Andre Shoow qui peut être considéré comme le 'numéro zéro' de la série, The first en 1995, Le second en 1997 et Quatre moins un en 1999) réalisés par Vincent Patar et Stéphane Aubier. Les personnages principaux sont Pic Pic, un cochon magique (ou 'magik') et André, un mauvais cheval. Bien qu'appartenant à la même série, ils ne se rencontrent jamais dans leurs aventures et vivent dans deux univers séparés (excepté dans le générique et un épisode bonus de quelques secondes).\\\",1,4,4,1988,\"|Action|Adventure|Animation|Comedy|Mini-Series|Western|\",\\\"\\\"\",
  "seasons" => "1",
  "@version" => "1",
  "description" => "Le Pic Pic André Shoow est une série de court métrages d'animation de quatre épisodes (Un premier en 1988 intitulé Picpic Andre Shoow qui peut être considéré comme le 'numéro zéro' de la série, The first en 1995, Le second en 1997 et Quatre moins un en 1999) réalisés par Vincent Patar et Stéphane Aubier. Les personnages principaux sont Pic Pic, un cochon magique (ou 'magik') et André, un mauvais cheval. Bien qu'appartenant à la même série, ils ne se rencontrent jamais dans leurs aventures et vivent dans deux univers séparés (excepté dans le générique et un épisode bonus de quelques secondes).\",
  "event" => {
    "original" => "8323,\"Pic Pic et André\",\\\"Le Pic Pic André Shoow est une série de court métrages d'animation de quatre épisodes (Un premier en 1988 intitulé Picpic Andre Shoow qui peut être considéré comme le 'numéro zéro' de la série, The first en 1995, Le second en 1997 et Quatre moins un en 1999) réalisés par Vincent Patar et Stéphane Aubier. Les personnages principaux sont Pic Pic, un cochon magique (ou 'magik') et André, un mauvais cheval. Bien qu'appartenant à la même série, ils ne se rencontrent jamais dans leurs aventures et vivent dans deux univers séparés (excepté dans le générique et un épisode bonus de quelques secondes).\\\",1,4,4,1988,\"|Action|Adventure|Animation|Comedy|Mini-Series|Western|\",\\\"\\\"\"
  },
  "creation_year" => "1988",
  "genres" => "|Action|Adventure|Animation|Comedy|Mini-Series|Western|",
  "network" => "",
  "show_id" => "8323",
  "title" => "Pic Pic et André",
  "episodes" => "4",
  "host" => {
    "name" => "nosql-virtual-machine"
  },
  "@timestamp" => 2023-11-13T18:31:36.280508279Z
}
```

En fonction des indications précédentes et du code fourni, intégrer également la transformation du second fichier.
Supprimer les champs `message`, `host` et `event` pour chaque fichier.

ATTENTION à l'ordre des champs dans `columns` du plugin `csv` qui doit correspondre à celui des fichiers sources (Cf. annexe) : bien vérifier que vous avez les mêmes valeurs que ci-dessus dans les champs (ex. : « YouTube » dans « `network` » et non dans un autre champ).

Résultats attendus :

- Séries (shows) :

```
{
  "@version" => "1",
  "seasons" => "1",
  "description" => "Two Missouri brothers, Taimoor and Rehan, travel the country taking on unconventional run-down structures and transforming them into surprising, ingenious dream homes.",
  "episodes" => "14",
  "genres" => "|Home and Garden|Reality|",
  "show_id" => "16078",
  "creation_year" => "2017",
  "network" => "FVI",
  "title" => "You Can't Turn That Into A House",
  "@timestamp" => 2023-11-13T19:22:56.128419366Z,
  "log" => {
    "file" => {
      "path" => "/home/nosql/elasticsearch/data/betaserie_shows.csv"
    }
  },
  "follow_count" => "0"
}

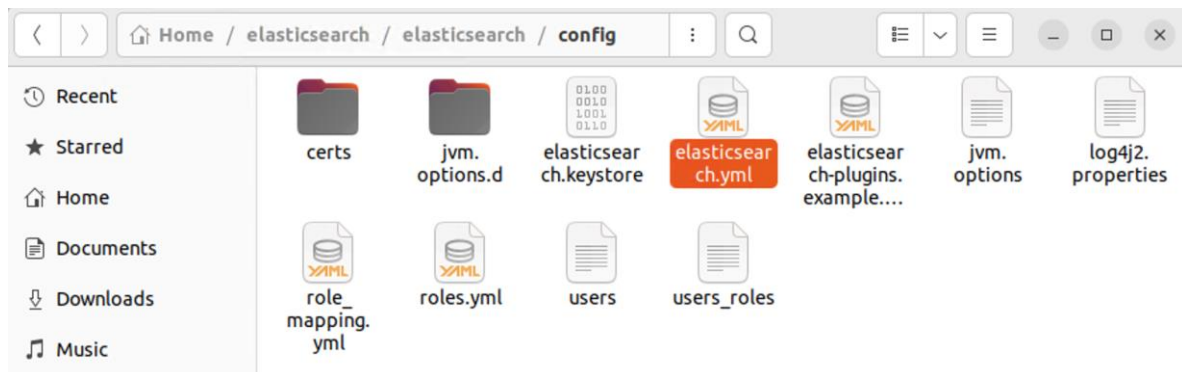
{
  "@version" => "1",
  "seasons" => "15",
  "description" => "Red vs. Blue centers on the Red and Blue Teams, two groups of soldiers engaged in a civil war. Each team occupies a small base in a box canyon known as Blood Gulch. According to Simmons (Gustavo Sorola), one of the Red Team soldiers, each team's base exists only in response to the other team's base. Although both teams generally dislike each other and have standing orders to defeat their opponents and capture their flag, neither team's soldiers are usually motivated to fight each other- if they are otherwise, neither are efficient. Teammates have an array of eccentric personalities and often create more problems for each other than for their enemies.",
  "episodes" => "340",
  "genres" => "|Animation|Comedy|",
  "show_id" => "5091",
  "creation_year" => "2003",
  "network" => "Rooster Teeth",
  "title" => "Red vs. Blue",
  "@timestamp" => 2023-11-13T19:22:56.128644078Z,
  "log" => {
    "file" => {
      "path" => "/home/nosql/elasticsearch/data/betaserie_shows.csv"
    }
  },
  "follow_count" => "106"
}
```

Capture d'écran

- Episodes :

```
"@timestamp" => 2023-11-13T19:13:45.085658552Z,
"released_timestamp" => "724546800",
"code" => "S01E06",
"show_id" => "52",
"description" => "Saffy prépare une fête surprise pour l'anniversaire d'Edina. La mère, Bubble, Justin (le père de Saffy), Oliver, Marshall et sa nouvelle femme Bo l'attendent alors mais elle refuse de se montrer car elle a 40 ans.",
"@version" => "1",
"episode_id" => "134696"
}
{
  "episode" => "13",
  "season" => "3",
  "show_title" => "You Gotta Eat Here!",
  "title" => "Fancy Franks, Soda Jerks, Pedal & Tap",
  "log" => {
    "file" => {
      "path" => "/home/nosql/elasticsearch/data/betaseries_episodes.csv"
    }
  },
  "@timestamp" => 2023-11-13T19:13:44.970520585Z,
  "released_timestamp" => "1400796000",
  "code" => "S03E13",
  "show_id" => "15990",
  "description" => "Host John Catucci fancy dresses beef franks, says 'Si, por favor' to the Macho Nacho Burger and accidentally discovers Meat & Spaghetti balls.",
  "@version" => "1",
  "episode_id" => "874681"
}
{
  "episode" => "14",
  "season" => "3",
  "show_title" => "You Gotta Eat Here!",
  "title" => "Pig BBQ Joint, Two Six (ate), Taco Pica",
  "log" => {
    "file" => {
      "path" => "/home/nosql/elasticsearch/data/betaseries_episodes.csv"
    }
  },
}
```

C'est le moment de démarrer Elasticsearch et Cerebro, un outil de requête et monitoring pour Elasticsearch. Nous allons d'abord désactiver la sécurité d'Elasticsearch (certificats). Editer le fichier elasticsearch.yml :



Passer à false les lignes suivantes :


```
elasticsearch.yml
~/elasticsearch/elasticsearch/config

83
84 #----- BEGIN SECURITY AUTO CONFIGURATION -----
85 #
86 # The following settings, TLS certificates, and keys have been automatically
87 # generated to configure Elasticsearch security features on 13-11-2023 17:39:19
88 #
89 #-----
90
91 # Enable security features
92 xpack.security.enabled: false
93
94 xpack.security.enrollment.enabled: false
95
96 # Enable encryption for HTTP API client connections, such as Kibana, Logstash, and Agents
97 xpack.security.http.ssl:
98   enabled: false
99   keystore.path: certs/http.p12
100
101 # Enable encryption and mutual authentication between cluster nodes
102 xpack.security.transport.ssl:
103   enabled: false
104   verification_mode: certificate
105   keystore.path: certs/transport.p12
106   truststore.path: certs/transport.p12
107 # Create a new cluster with the current node only
108 # Additional nodes can still join the cluster later
109 cluster.initial_master_nodes: ["nosql-virtual-machine"]
110
```

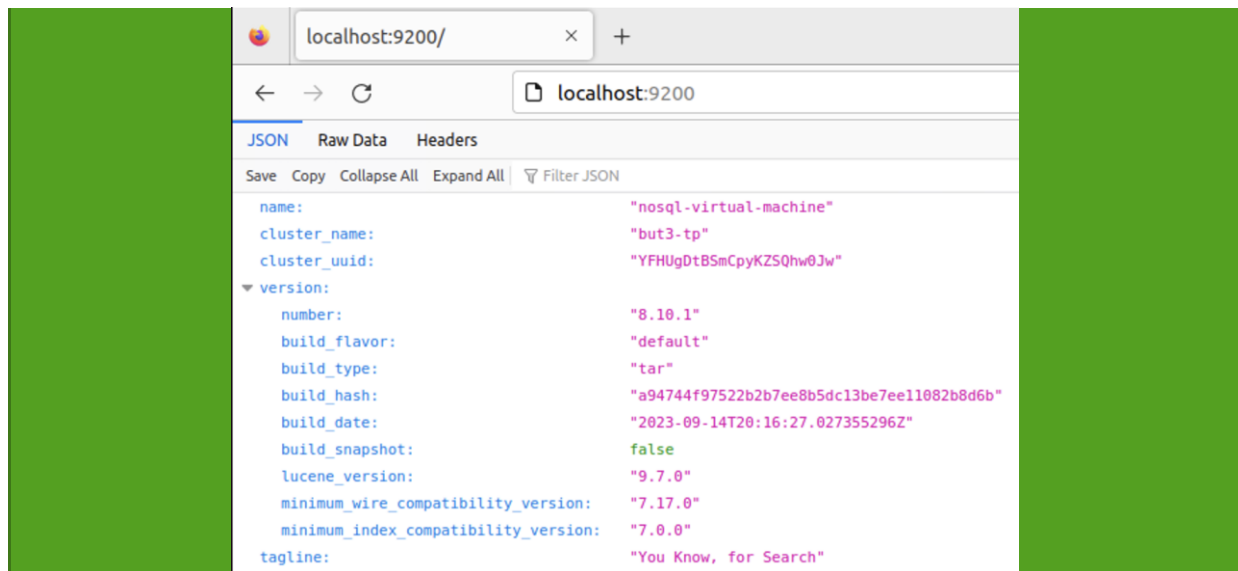
Sauvegarder.

Il suffit ensuite d'exécuter le script `sh start.sh` (dans un autre terminal) pour les démarrer tous les deux.

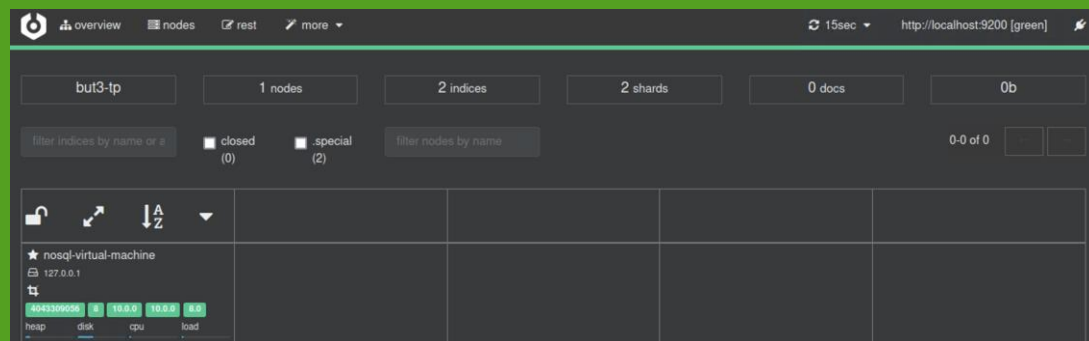
```
nosql@nosql:~/elasticsearch$ sh start.sh
#####
### Launching ES & Cerebro ###
#####
nosql@nosql:~/elasticsearch$ WARNING: An illegal reflective access operation has occurred
WARNING: Illegal reflective access by com.google.inject.internal.cglib.core.$ReflectUtils$1 (file:
/home/nosql/elasticsearch/cerebro/lib/com.google.inject.guice-4.2.3.jar) to method java.lang.Class
Loader.defineClass(java.lang.String,byte[],int,int,java.security.ProtectionDomain)
WARNING: Please consider reporting this to the maintainers of com.google.inject.internal.cglib.cor
e.$ReflectUtils$1
WARNING: Use --illegal-access=warn to enable warnings of further illegal reflective access operati
ons
WARNING: All illegal access operations will be denied in a future release
[info] play.api.Play - Application started (Prod) (no global state)
[info] p.c.s.AkkaHttpServer - Listening for HTTP on /0:0:0:0:0:0:0:0:9000
```

Pour vérifier que tout a démarré :

- Se rendre sur [HTTP://localhost:9200](http://localhost:9200) -> C'est l'adresse d'ElasticSearch. Vérifier que le nom du cluster est bien « but3-tp »



- Se rendre sur [HTTP://localhost:9000](http://localhost:9000) et entrer l'adresse d'ElasticSearch pour se connecter au cluster

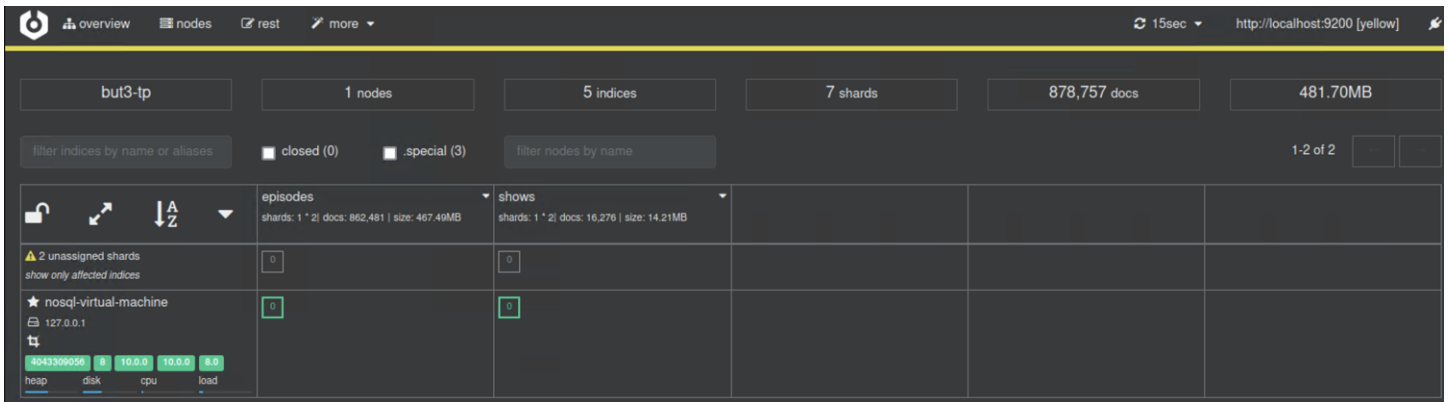


- c) Il est maintenant temps d'envoyer nos données formatées sur Elasticsearch. Nous allons donc faire deux index (indices) : *shows* et *episodes*. Remplacer le plugin stdout par un plugin elasticsearch. Vu que Logstash traite nos fichiers CSV en même temps, il faut qu'on différencie les sorties : épisodes vers un index « episodes », séries vers un index « shows » (utiliser index dans le plugin elasticsearch). Comme précédemment, utiliser un if pour cela.

Documentation du plugin elasticsearch : <https://www.elastic.co/guide/en/logstash/8.10/plugins-outputs-elasticsearch.html>

Réutiliser le même IF que pour le plugin CSV mais en mettant une des sorties elasticsearch à la place.

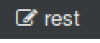
Résultat attendu : Sur Cerebro, il y a maintenant deux index visibles (*shows* et *episodes*) avec des documents à l'intérieur (16 276 séries et 862 481 épisodes) sur la partie « overview ». Il faudra attendre un peu, le temps de charger tous les épisodes, **surtout les derniers...**



Maintenant que toutes les données sont dans Elasticsearch, vous pouvez fermer Logstash et ses dossiers.

Intégrer le code final Logstash (question c) à votre compte-rendu.

2. Query Elasticsearch

Cerebro permet d'exécuter des requêtes REST directement sur Elasticsearch via l'onglet « rest » .

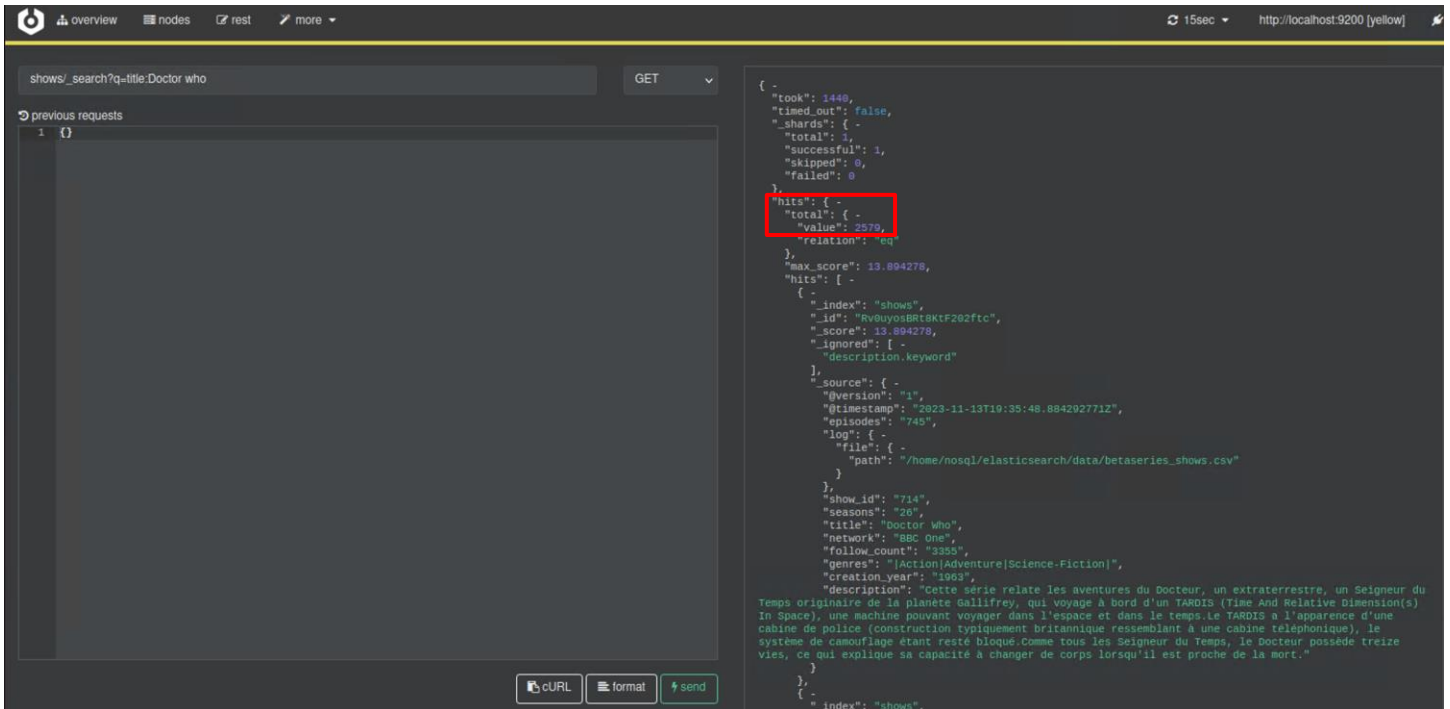
Les requêtes Elasticsearch s'effectuent en HTTP sur des *Endpoint* REST qui expriment l'index cible et l'action à réaliser (CRUD). Autrement dit, on effectue de simples requêtes HTTP sur un chemin particulier avec la recherche en paramètre ou dans le corps de celle-ci.

Rappel sur le REST HTTP et un CRUD :
POST -> Création d'objet dans le corps de l'appel (**CREATE**)
GET -> Affiche (**READ**)
PUT -> mise à jour (**UPDATE**)
DELETE -> Suppression (**DELETE**)
 plus de détails ici : <https://blog.nicolashachet.com/niveaux/confirmelarchitecture-rest-expliquee-en-5-regles/>

Les chemins Elasticsearch se décomposent de cette façon : {index}/{id} ou {index}/{action}. L'action pour faire des recherches est `_search`. Ainsi pour rechercher dans les épisodes, le chemin sera `episodes/_search`

Il existe deux systèmes de requêtes sur Elasticsearch : la *Query DSL* et la *Query String*. On va commencer par cette dernière.

Une query string s'effectue en GET sur `shows/_search` avec en paramètre `q=MA_QUERY`. **Exemple :** `GET shows/_search?q=title:Doctor who`. Dans l'onglet « rest », on obtient 2579 enregistrements :



Pour voir le format de ces requêtes, vous pouvez vous reporter au cours ou à la documentation d'Elasticsearch (<https://www.elastic.co/guide/en/elasticsearch/reference/8.10/query-dsl-query-string-query.html#query-string-syntax>)

a) Réaliser ces requêtes :

- Les séries qui ont dans le titre le mot « detective » (afficher tous les résultats) -> 37 résultats
- Les séries qui ont dans le titre le mot « detective » et qui ont plus de 3 saisons -> 4 résultats
- Les séries qui ont dans le titre le mot « batman » mais sans le mot pingouin dans la description -> 9 résultats
- Les Séries qui ont dans la description les mots « alien » ou « robot » du genre « Comedy » et qui ont été créés dans les années 90 -> 5 résultats

ATTENTION, les opérateurs booléens LUCENE s'écrivent en majuscules

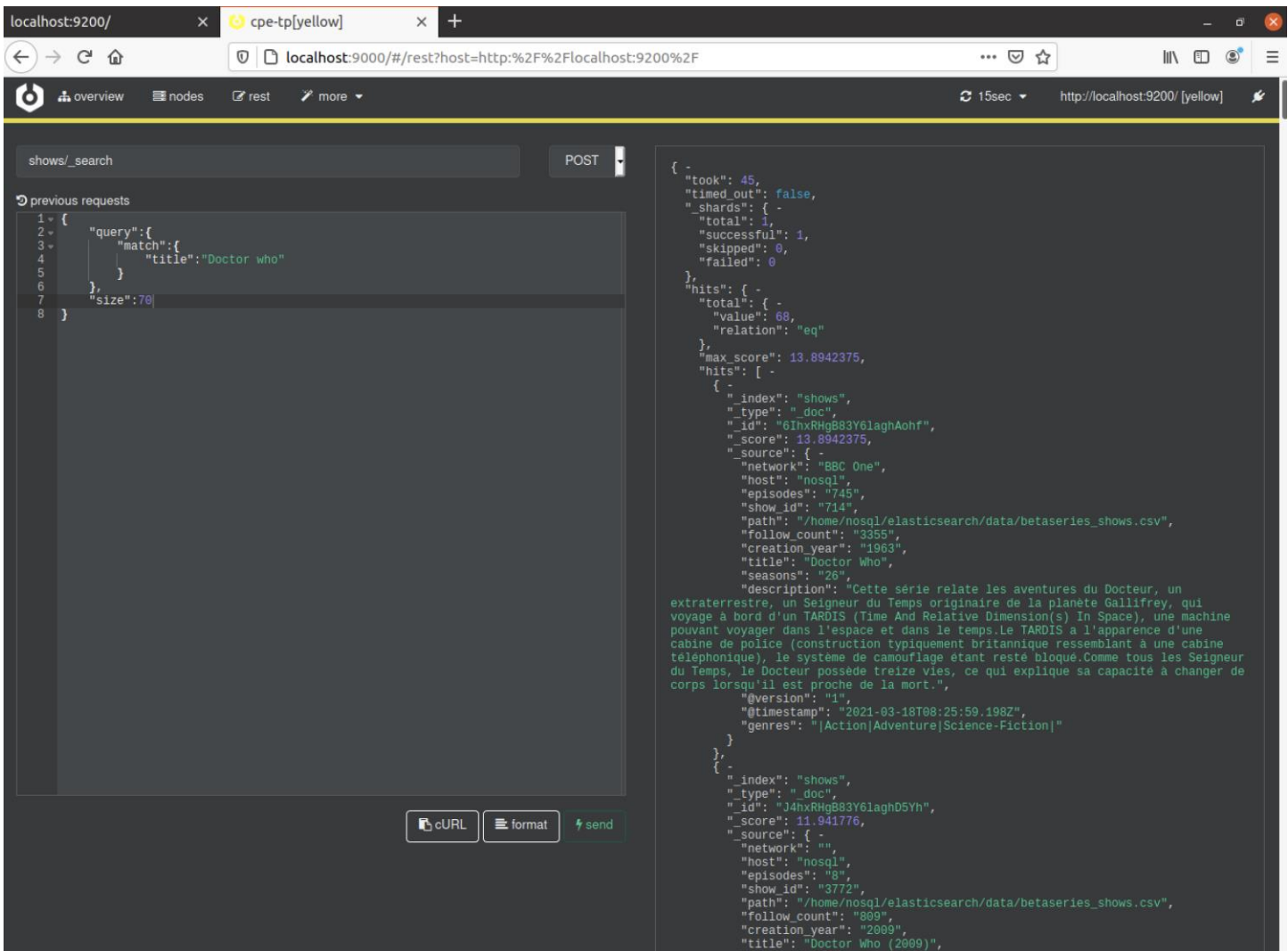
Intégrer le code et les copies d'écran de ces requêtes à votre compte-rendu.

Nous allons maintenant passer au query DSL.

La Query DSL s'effectue sur le même chemin que les query string mais sans aucun paramètre. Toute la requête sera contenue dans un JSON dans le corps de l'appel HTTP qui est envoyé via un POST.

Le AND et OR deviennent une bool query et les échelles deviennent des range query. Le chapitre de la documentation sur la recherche full text vous donne toutes les clés pour comprendre le système de query DSL (<https://www.elastic.co/guide/en/elasticsearch/reference/8.10/query-dsl.html>).

Exemple : recherche exacte sur le titre "Doctor who"



- b) Essayer maintenant de faire les requêtes précédentes en Query DSL.
Intégrer le code et les copies d'écran de ces requêtes à votre compte-rendu.
- c) Essayer de faire une recherche sur les séries Star Trek en triant par nombres d'épisodes (champ episodes dans les documents shows)
ATTENTION : cela devrait normalement vous indiquer une erreur : les « fielddata » ne sont pas activés pour les champs texte (Cf. écran suivant). En effet, il n'est pas possible d'ordonner sur un champ de type texte sans fielddata : <https://www.elastic.co/guide/en/elasticsearch/reference/8.10/text.html#fielddata-mapping-param>

```
{
  "error": {
    "root_cause": [
      {
        "type": "illegal_argument_exception",
        "reason": "Fielddata is disabled on [episodes] in [shows]. Text fields are not optimised for operations that require per-document field data like aggregations and sorting, so these operations are disabled by default. Please use a keyword field instead. Alternatively, set fielddata=true on [episodes] in order to load field data by uninverting the inverted index. Note that this can use significant memory."
      }
    ],
    "type": "search_phase_execution_exception",
    "reason": "all shards failed",
    "phase": "query",
    "grouped": true,
    "failed_shards": [
      {
        "shard": 0,
        "index": "shows",
        "node": "g770IA39T0aEWg1kDsV9fA",
        "reason": {
          "type": "illegal_argument_exception",
          "reason": "Fielddata is disabled on [episodes] in [shows]. Text fields are not optimised for operations that require per-document field data like aggregations and sorting, so these operations are disabled by default. Please use a keyword field instead. Alternatively, set fielddata=true on [episodes] in order to load field data by uninverting the inverted index. Note that this can use significant memory."
        }
      }
    ],
    "caused_by": {
      "type": "illegal_argument_exception",
      "reason": "Fielddata is disabled on [episodes] in [shows]. Text fields are not optimised for operations that require per-document field data like aggregations and sorting, so these operations are disabled by default. Please use a keyword field instead. Alternatively, set fielddata=true on [episodes] in order to load field data by uninverting the inverted index. Note that this can use significant memory."
    },
    "caused_by": {
      "type": "illegal_argument_exception",
      "reason": "Fielddata is disabled on [episodes] in [shows]. Text fields are not optimised for operations that require per-document field data like aggregations and sorting, so these operations are disabled by default. Please use a keyword field instead. Alternatively, set fielddata=true on [episodes] in order to load field data by uninverting the inverted index. Note that this can use significant memory."
    }
  },
  "status": 400
}
```

Vérifier le mapping de l'index shows pour voir le type du champ « episodes » (GET _all/_mapping pour voir les 2 mappings ou GET shows/_mapping pour voir le mapping de shows)

The screenshot shows the Kibana interface for the Elasticsearch index 'shows'. The 'shows/_mapping' endpoint is selected, and the mapping is displayed. The mapping includes the following fields:

- timestamp**: type: date
- creation_year**: type: text, fields: keyword (ignore_above: 256)
- description**: type: text, fields: keyword (ignore_above: 256)
- episodes**: type: text, fields: keyword (ignore_above: 256)
- follow_count**: type: text, fields: keyword (ignore_above: 256)

The 'episodes' field is highlighted in blue. The bottom bar shows 'cURL', 'format', and 'send' buttons.

Lorsque que Logstash indexe les données sur Elasticsearch, il applique un mapping par défaut qui considère tous les champs comme des champs texte avec un sous-champs analysé. On peut définir un mapping personnalisé dans le plugin output de Logstash mais nous allons plutôt faire une réindexation directement dans Elasticsearch.

Intégrer les copies d'écran des 2 mappings à votre compte-rendu.

3. Réindexation et Mapping

Nous allons donc réindexer nos 2 index grâce à l'API Reindex. L'API Reindex s'occupe de transférer nos données vers un nouvel index existant, le plus souvent avec un mapping différent. Elasticsearch s'occupe dans ce cas-là de migrer le type des données. Le principe des mappings est expliqué ici : <https://www.elastic.co/guide/en/elasticsearch/reference/8.10/mapping.html>

- a) Compléter l'index shows (ci-dessous) en vous inspirant du mapping de l'index episodes-v2 (ci-dessous). Pour rappel, la création d'un index se fait avec un PUT <indexName> avec en payload le mapping de l'index.

Mapping episodes-v2 :

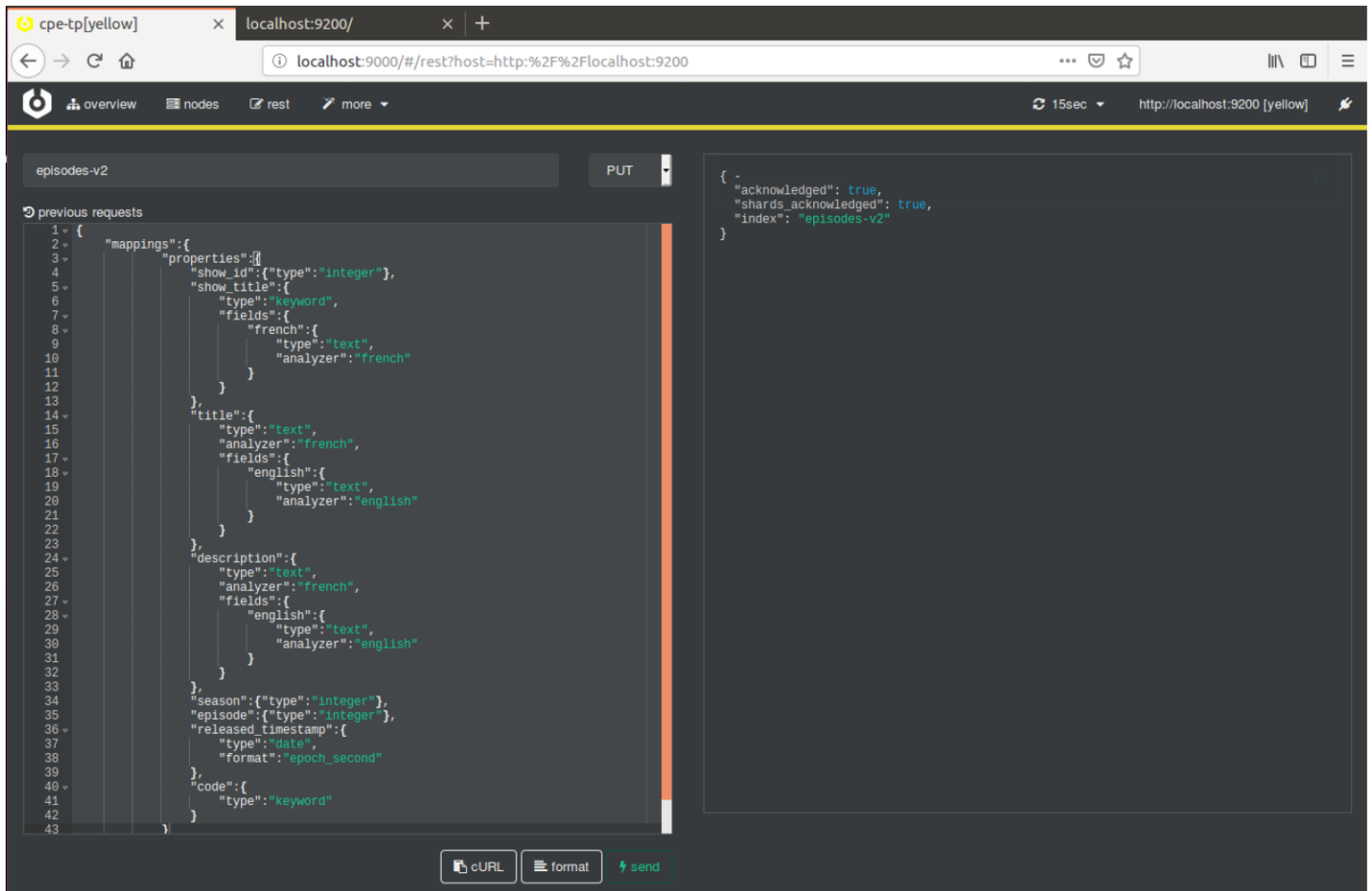
Pour les types choisis, voir annexe.

```
PUT episodes-v2
{
  "mappings":{
    "properties":{
      "episode_id":{"type":"integer"},
      "show_id":{"type":"integer"},
      "show_title":{
        "type":"keyword",
        "fields":{
          "french":{
            "type":"text",
            "analyzer":"french"
          }
        }
      },
      "title":{
        "type":"text",
        "analyzer":"french",
        "fields":{
          "english":{
            "type":"text",
            "analyzer":"english"
          }
        }
      },
      "description":{
        "type":"text",
        "analyzer":"french",
        "fields":{
          "english":{
            "type":"text",
            "analyzer":"english"
          }
        }
      },
      "season":{"type":"integer"},
      "episode":{"type":"integer"},
      "released_timestamp":{
        "type":"date",
        "format":"epoch_second"
      },
      "code":{
```

```

    "type": "keyword"
  }
}
}

```



Mapping shows-v2 (A COMPLETER et A CREER) :

Indications :

- *title* : comme *show_title* d'*episodes-v2*
- *description* : idem *description* d'*episodes-v2*
- Pour les autres champs, voir Annexe.

```

{
  "mappings":{
    "properties":{
      "show_id":{
        "type":"integer"
      },
      "genres":{
        "type":"text",
        "analyzer":"french",
        "fields":{
          "analyzed":{

```

```

        "type": "text",
        "fielddata": true,
        "analyzer": "simple"
      }
    },
  }
}

```

Intégrer le code du mapping de shows-v2 à votre compte-rendu.

- b) Quelle est la différence entre un champ de type Keyword et un champ de type Text ?

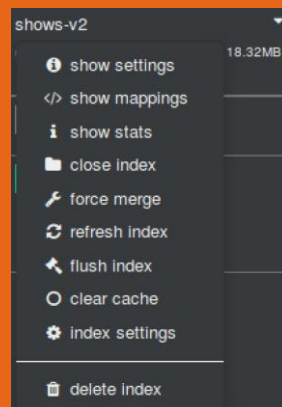
*Pour les champs analysés, il faut ajouter un sous-champ aux champs de type text avec un analyzer français et un sous champ analyzer anglais
Exemple dans la documentation avec un analyzer anglais :
<https://www.elastic.co/guide/en/elasticsearch/reference/8.10/docs-reindex.html>*

- c) Grâce à l'endpoint de réindexation (<https://www.elastic.co/guide/en/elasticsearch/reference/8.10/docs-reindex.html>), migrer les données vers nos nouveaux indexes.

Il se peut qu'il reste un document dans l'ancien index qui ne corresponde pas au mapping que nous venons de faire. Dans ce cas-là, Elasticsearch renvoie une erreur et arrête la réindexation. Si vous avez cette erreur, garder l'ID du document (donné dans le retour) et supprimer ce document dans l'index d'origine. Il faudra par la suite supprimer l'index v2 (vous pouvez aussi le faire dans l'onglet Overview de Cerebro), le recréer et relancer la ré-indexation.

- Suppression d'un document, exemple : `DELETE shows/_doc/<ID>` :
<https://www.elastic.co/guide/en/elasticsearch/reference/8.10/docs-delete.html>

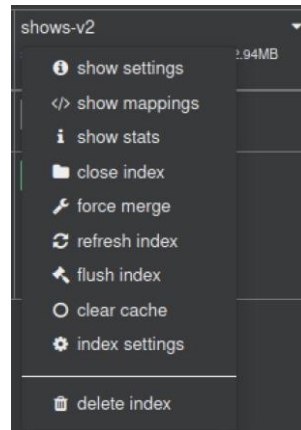
- Suppression d'un index dans Cerebro (« delete index ») :



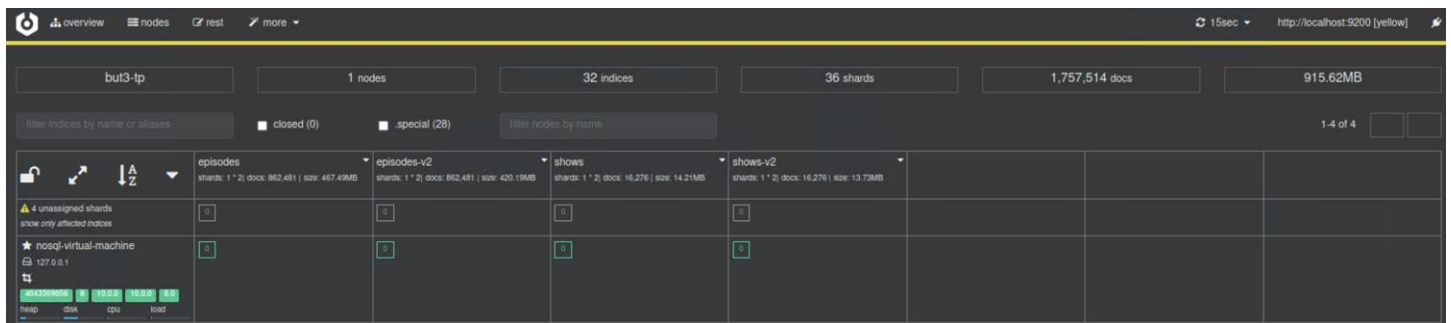
COMMENCER PAR REINDEXER SHOWS PUIS EPISODES (sera très long). Ne pas la relancer sinon DOUBLONS (sinon supprimer l'index, recréer le mapping et relancer la réindexation)

Ajouter les copies d'écran des appels REST à votre compte-rendu.

Mettre à jour les statistiques (nombre de documents, taille) d'un index dans Cerebro : « refresh index »



Résultat (après refresh) :



- d) Ré-exécuter les requêtes de la partie II b). Le 3 premières requêtes Query DSL ne renvoient normalement plus de résultat. Essayer, par exemple la requête n°1, avec une recherche exacte sur le titre (« True Detective »). Compléter votre réponse de la question b).
- e) Retenter la requête des séries star trek de la partie II c). Cette fois-ci la liste triée devrait s'afficher :

```
{
  "took": 291,
  "timed_out": false,
  "_shards": {
    "total": 1,
    "successful": 1,
    "skipped": 0,
    "failed": 0
  },
  "hits": {
    "total": {
      "value": 1,
      "relation": "eq"
    },
    "max_score": null,
    "hits": [
      {
        "_index": "shows-v2",
        "_type": "_doc",
        "_id": "8ohxRHg883Y6lghAohf",
        "_score": null,
        "_source": {
          "network": "NBC",
          "host": "nosql",
          "episodes": "80",
          "show_id": "723",
          "path": "/home/nosql/elasticsearch/data/betaseres_shows.csv",
          "follow_count": "2599",
          "creation_year": "1966",
          "title": "Star Trek",
          "seasons": "3",
          "description": "Cette série raconte les aventures vécues, au XXIIIe siècle, par James T. Kirk, capitaine du vaisseau Enterprise NCC-1701 et son équipage. Leur mission quinquennale est d'explorer la galaxie afin d'y découvrir d'autres formes de vie et d'enrichir ainsi les connaissances humaines.",
          "@version": "1",
          "@timestamp": "2021-03-18T08:25:59.205Z",
          "genres": "Action|Adventure|Drama|Science-Fiction|"
        }
      }
    ]
  },
  "sort": [
    80
  ]
}
```

- f) Tester l'analyser français en regardant comment il interprète des recherches en langue française (GET `_analyze`). Exemple de phrase à analyser : « Comment réparer un vélo avec une brosse à dent ? »

C'est cette analyse qui permet de calculer le score sur nos recherches. En le mettant en français, l'analyser est ainsi plus à même de comprendre nos recherches et de retrouver des résultats concluants

Intégrer vos tests et les copies d'écran de ces requêtes à votre compte-rendu.

4. Agrégations

Nous allons maintenant faire quelques agrégations pour recueillir quelques statistiques sur nos données. Utiliser les index « v2 » que nous avons créés précédemment.

Faire ces différentes requêtes (mettre `size = 0` dans la requête pour que ce soit plus rapide et lisible dans le résultat) :

- Donner le nombre moyen de saisons par séries, ainsi que le nombre moyen de followers. Résultat : 720.89 de followers moyens et 2.31 saisons en moyenne
- Donner le nombre moyen de followers pour les séries « Doctor who ». Résultat : 746.93
- Donner le nombre moyen de followers par année de création des séries de 1968 à 2018. Exemples pour 1968 : 102.93 ; 1969 : 60.56 ; etc.
- Donner les 3 networks ayant le plus de séries entre 1900 et 2018 et pour chacun, le nombre de séries créées par année. Résultats : Youtube en 2018 -> 4, en 2017 -> 85, ..., en 2003 -> 1, ...
- Compter le nombre de séries par tranche de saisons (1-2, 2-3, 4-6, 7+). Indice : *range aggregation*. Résultats : entre 2 et 3 : 2386 ; entre 4 et 6 : 1184, etc.
- Donner les 5 genres les plus populaires par année depuis 2017. Indications : il faudra accéder au champ *analyzed de genres*. En 2017 : drama 496, comedy 365 ... En 2018 : drama 67, animation 39, comedy 39,...

Intégrer le code et les copies d'écran de ces requêtes à votre compte-rendu.

Documentation :

- Doc FR : <https://netapsys.developpez.com/tutoriels/nosql/agregations-elasticsearch/>
- Doc US : <https://logz.io/blog/elasticsearch-aggregations/>
- Doc officielle : <https://www.elastic.co/guide/en/elasticsearch/reference/8.10/search-aggregations.html>

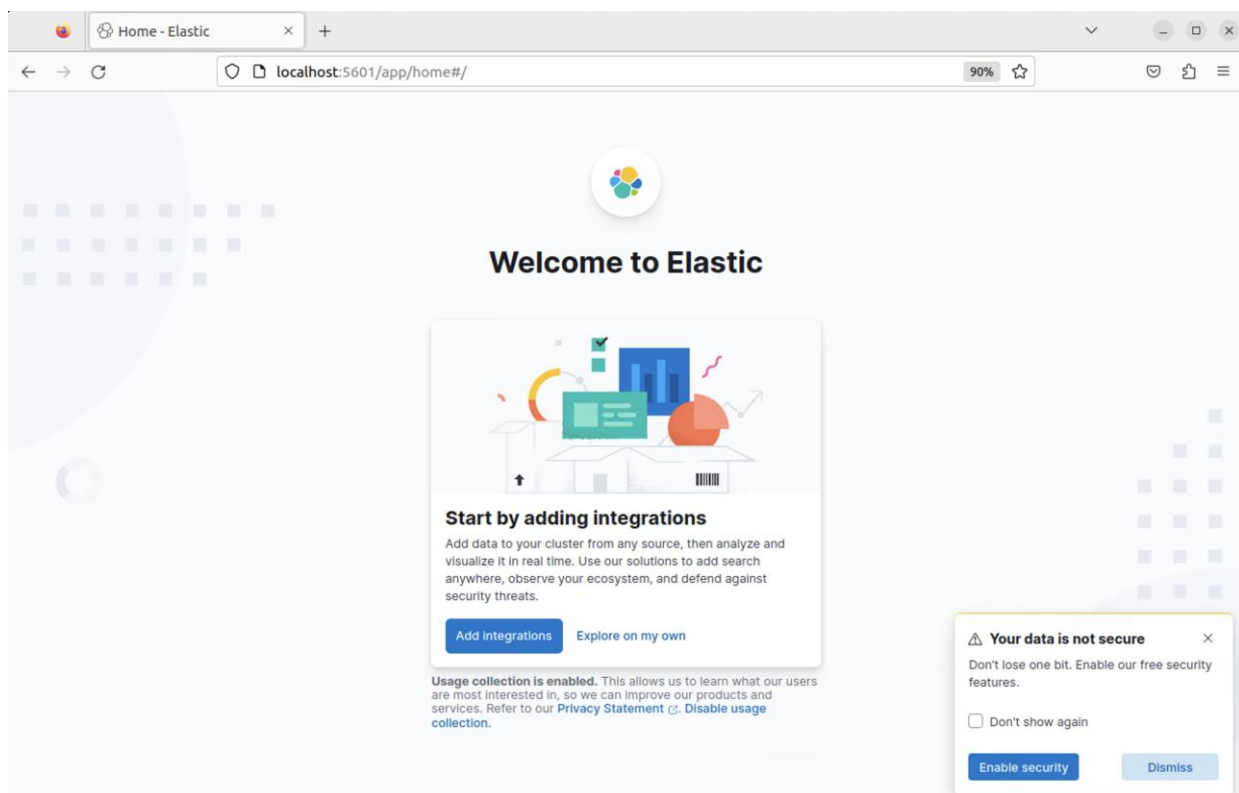
5. Kibana

Kibana est l'outil de visualisation (dataviz) des données d'Elasticsearch. C'est une façon simple d'explorer et d'afficher les données d'Elasticsearch. Il y a trois onglets principaux :

- Discover : Permet de fouiller dans la donnée en ajoutant des filtres (Query)
- Visualize : Faire des graphiques interactifs sur des agrégations
- Dashboard : assemblage de graphiques pour faire des tableaux de bord

Pour lancer Kibana, se rendre dans le dossier Kibana et exécuter la commande `bin/kibana &`

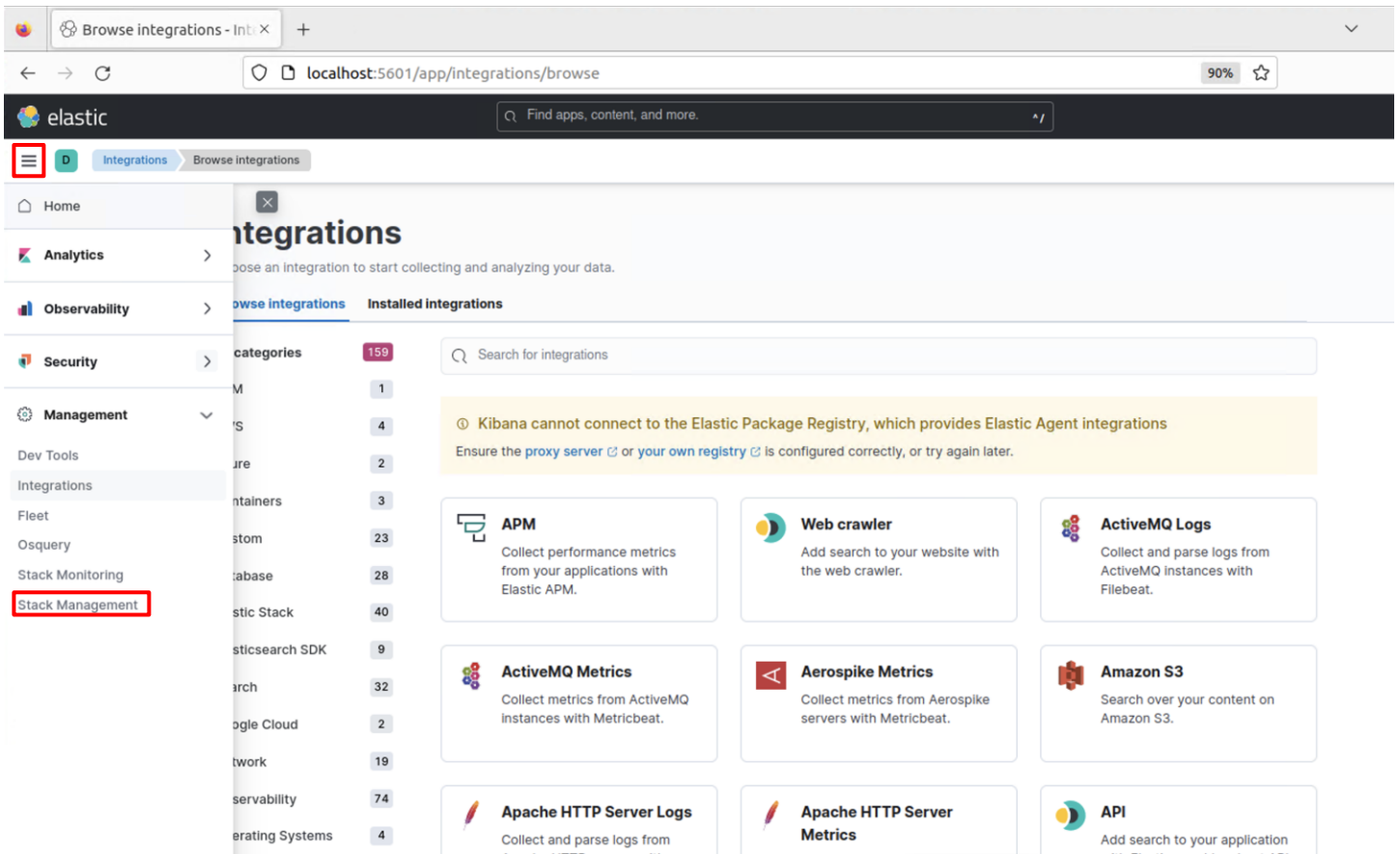
URL de Kibana : <http://localhost:5601>



Cliquer sur le bouton « Dismiss » pour ne pas activer la sécurité.

Cliquer ensuite sur le bouton « Add Integrations ». Il va alors se connecter au cluster ES sur le port 9200, par défaut.

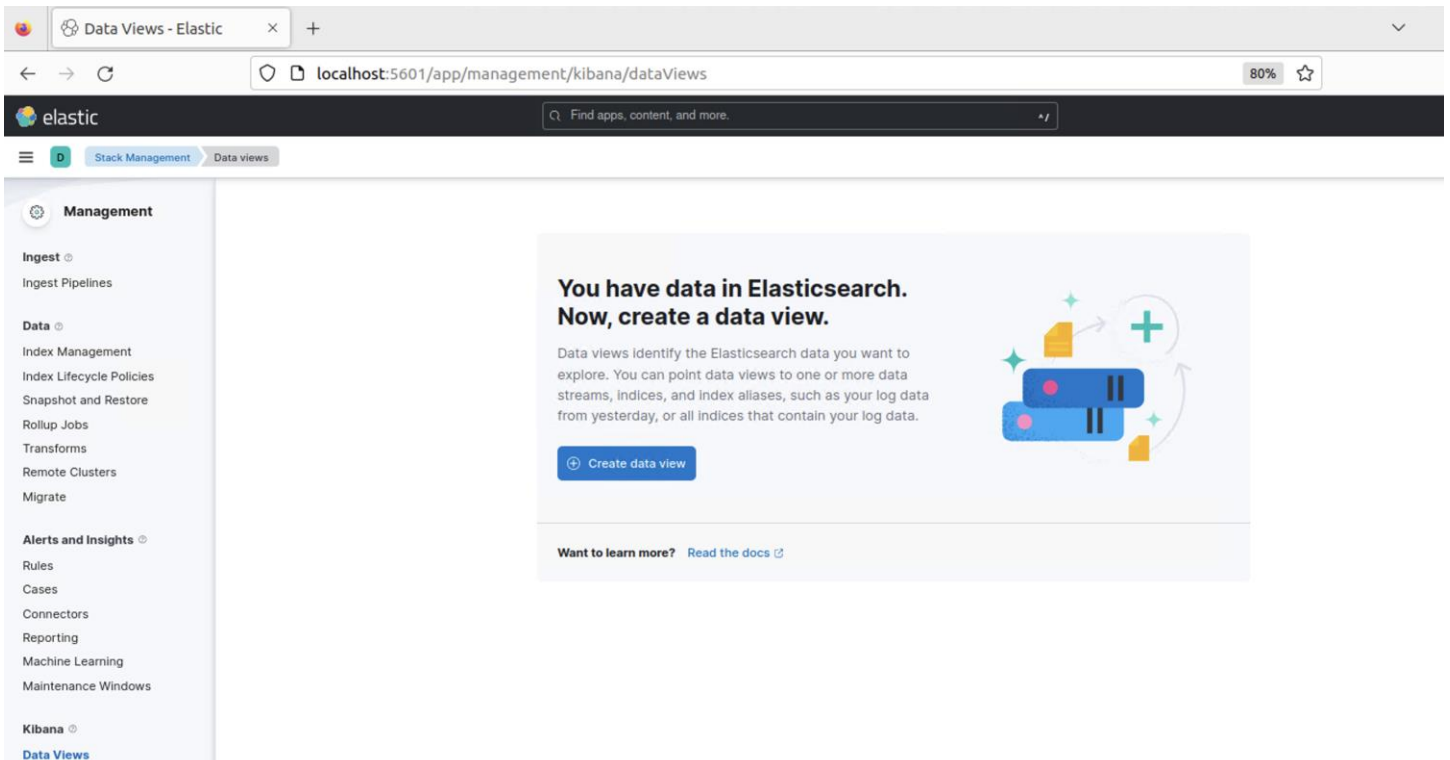
Cliquer sur le menu général puis menu *Management* > *Stack management*.



Il faut ensuite paramétrer un « Data View » incluant l'index « shows-v2 » et un autre pour « episodes-v2 ».

Exemple : episodes-v2 :

Menu *Data Views* puis bouton *Create data view*



Ne pas appliquer de filtre par défaut :

Create data view

Name

episodes-v2

Index pattern

episodes-v2*

Timestamp field

--- I don't want to use the time filter ---

Select a timestamp field for use with the global time filter.

Show advanced settings

✓ Your index pattern matches 1 source.

All sources

Matching sources

episodes-v2

Index

Rows per page: 10

Close

Save data view to Kibana

episodes-v2

DeleteEdit

Index pattern: episodes-v2*

★ Default

Fields (21)

Scripted fields (0)

Field filters (0)

Relationships (0)

Search

Field type 7

Schema type 2

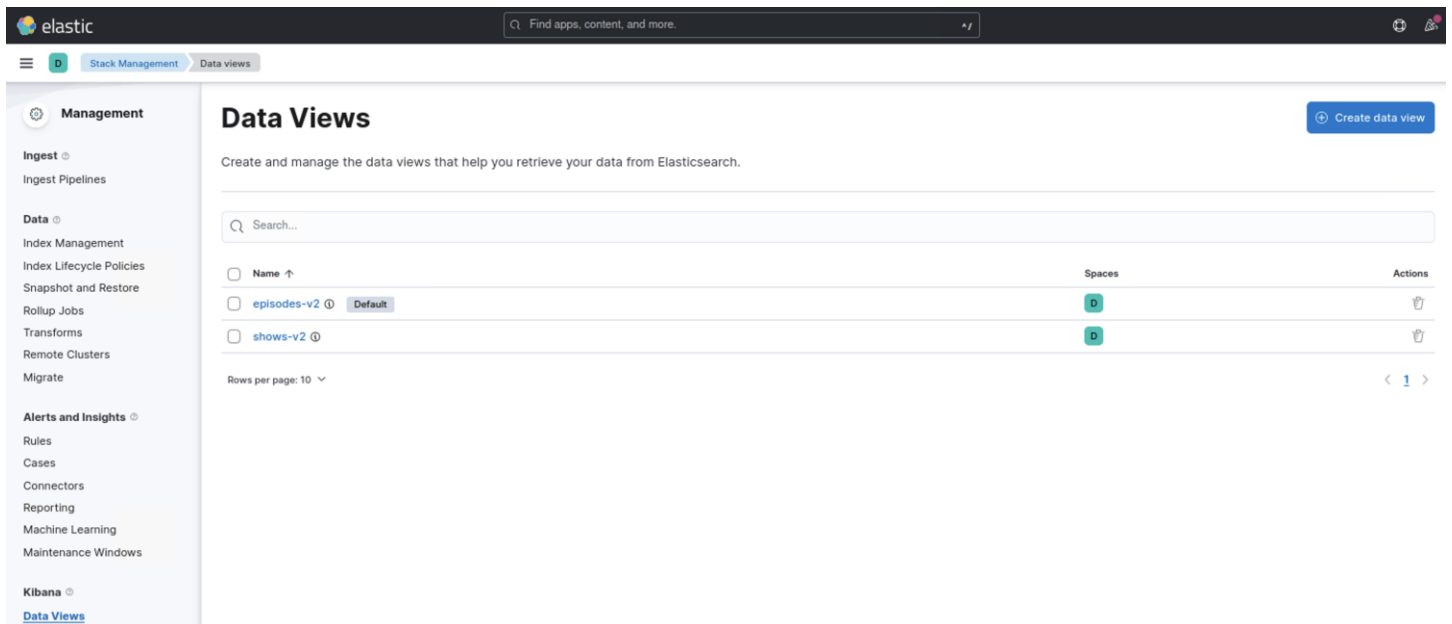
Add field

Name ↑	Type	Format	Searchable	Aggregatable	Excluded	
@timestamp	date		•	•		✎
@version	text		•			✎
@version.keyword	keyword		•	•		✎
_id	_id		•			✎
_index	_index		•	•		✎
_score						✎
_source	_source					✎
code	keyword		•	•		✎
description	text		•			✎
description.english	text		•			✎

Rows per page: 10

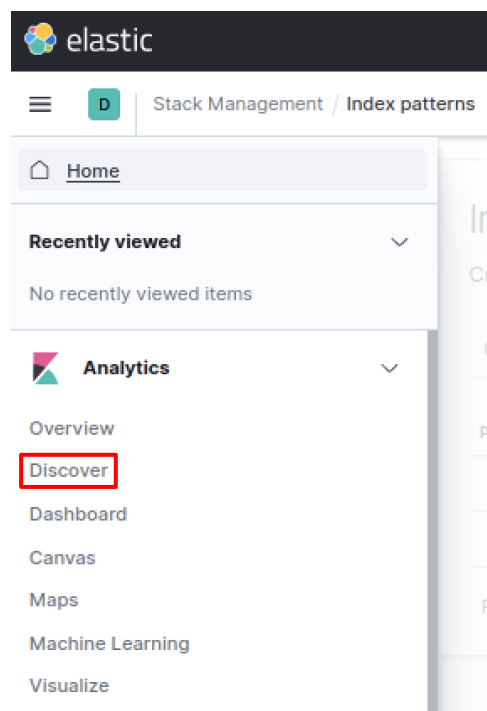
< 1 2 3 >

Idem pour shows-v2.



1) Reproduire les requêtes de la section 2 sur l'onglet Discover de Kibana

Cliquer sur « Discover »



The screenshot shows the Elastic Kibana interface. At the top, there's a search bar with the text 'Find apps, content, and more.' Below it, a navigation bar with 'Discover' and 'Save' buttons. The main area is divided into a sidebar on the left and a main content area on the right. The sidebar contains a search bar for field names and a list of available fields: @timestamp, @version, creation_year, description, episodes, follow_count, genres, log.file.path, network, seasons, show_id, and title. The main content area shows 16,276 hits. A table of results is displayed with columns for document ID, timestamp, and description. The results include various TV shows and their details.

Documentation Kibana :

- Tutoriels FR : <https://www.ionos.fr/digitalguide/web-marketing/analyse-web/tutoriel-kibana/>, https://www.ademec.com/sites/default/files/tutoriel_kibana.pdf
- Doc officielle : <https://www.elastic.co/guide/en/kibana/8.10/index.html>

Kibana Query Language : <https://www.elastic.co/guide/en/kibana/8.10/kuery-query.html>

Intégrer les copies d’écran de ces requêtes à votre compte-rendu.

2) Essayer de faire des graphiques en reproduisant les requêtes de la section 4.

Intégrer les copies d’écran à votre compte-rendu.

ANNEXES

STRUCTURE DES DOCUMENTS « SHOWS »

Champ	Type	Analysé ?
show_id	Integer	
title	Keyword	Oui (en sous-champ)
description	Text	oui
seasons	Integer	
episodes	Integer	
follow_count	Integer	
creation_year	Date (year)	
genres	Text	Oui <- simple analyzer
network	Keyword	non

STRUCTURE DES DOCUMENTS « ÉPISODES »

Champ	Type	Analysé ?
episode_id	Integer	
show_id	Integer	
show_title	Keyword	Oui (en sous-champ)
code	Keyword	non
season	Integer	
episode	Integer	
title	Text	oui
description	Text	oui
released_timestamp	Date (epoch_second)	